AISW수학

- 로지스틱 회귀 (1) -

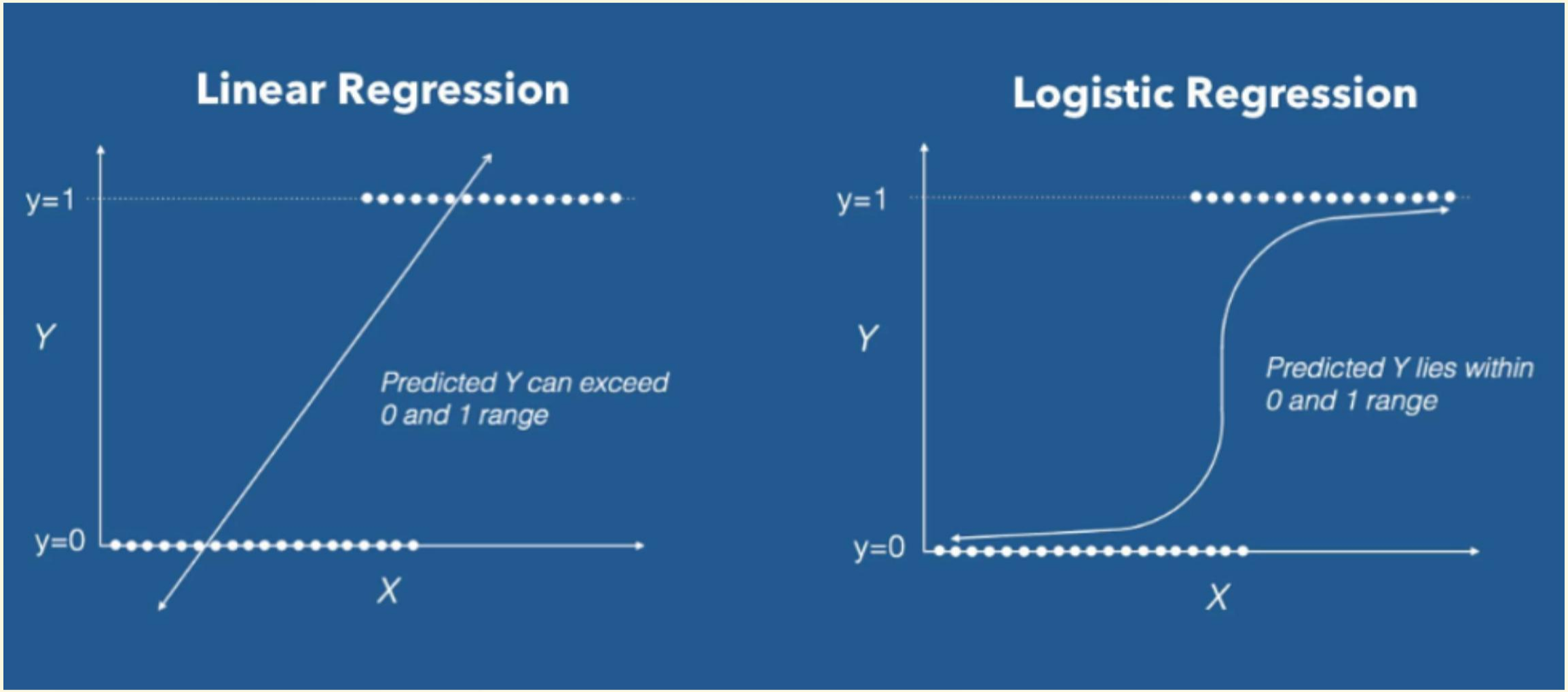
김 선 만

선형 회귀 (리마인드)

- 원인(독립변수)의 따라 결과(종속변수)가 발생함
  1. 공부한 시간 → 성적
  2. 자동차 속도 → 제동거리
  3. 키(height) → 몸무게(weight)
- 독립변수가 종속변수에 미치는 영향력의 크기를 파악하고, **독립변수에 따라 종속변수의 값을 예측하는 것을 회귀 분석**이라고 함
- 선형 회귀(Linear Regression)은 회귀 분석에서도 **독립변수에 따라 종속변수의 값이 선형**일 경우를 가리킴

로지스틱 회귀

- 원인(독립변수)의 따라 결과(종속변수)가 발생함
  1. 공부한 시간 → 과목통과 여부(Pass/Fail)
  2. 자동차 속도 → 과속 여부
  3. 키, 몸무게 → 과체중 여부
- **독립변수에 따라 종속변수의 범주(category)를 예측하는 것을 로지스틱 회귀**라고 함 - 이진 분류(Binary Classification) - 다중 분류(Multi-class Classification)


Linear Regression
y=1
Y
y=0
X
Predicted Y can exceed
0 and 1 range
Logistic Regression
y=1
Y
y=0
X
Predicted Y lies within
0 and 1 range

선형 회귀를 통한 이진 분류

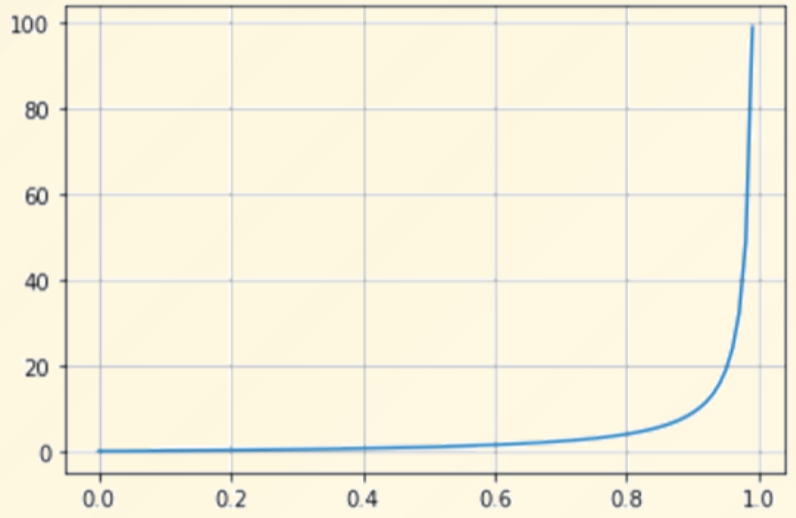
- 공부한 시간 → 과목통과 여부(Pass/Fail)
- 선형 회귀를 통하여 0이면 불합격(Fail), 1이면 합격(Pass)?

Q1) 0.5의 값으로 예측이 되면?
Q2) 0보다 작은 값으로 예측이 되면?
Q3) 1보다 큰 값으로 예측이 되면?

→ **0~1사이의 값으로 예측되는 결과가 필요**함!
→ **결과가 확률**로 나와야겠다! ex)합격할 확률은 0.7 (70%)

Odds

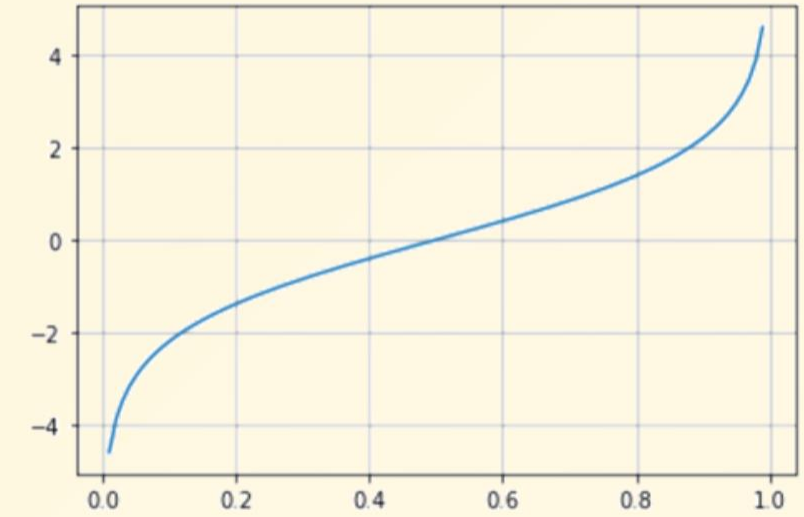
- P(X) : 사건이 일어날 확률
- 1 – P(X) : 사건이 일어나지 않을 확률
- 0 ≤ P(X) ≤ 1
- Odds: 해당 사건이 일어날 확률과 일어나지 않을 확률의 비율
- $\frac{P(X)}{1-P(X)}$



log-Odds

- **Odds에 log를 씌움 (logit)**

$$
\begin{aligned}
logit &= \ln\left(\frac{P(X)}{1-P(X)}\right) \\
&= \ln(P(X)) - \ln(1-P(X)) \\
&= -\ln(\frac{1}{p}-1)
\end{aligned}
$$



$$\rightarrow y = -\ln(\frac{1}{z}-1)$$

log-Odds

- **Inverse of logit function (Logistic Function)**

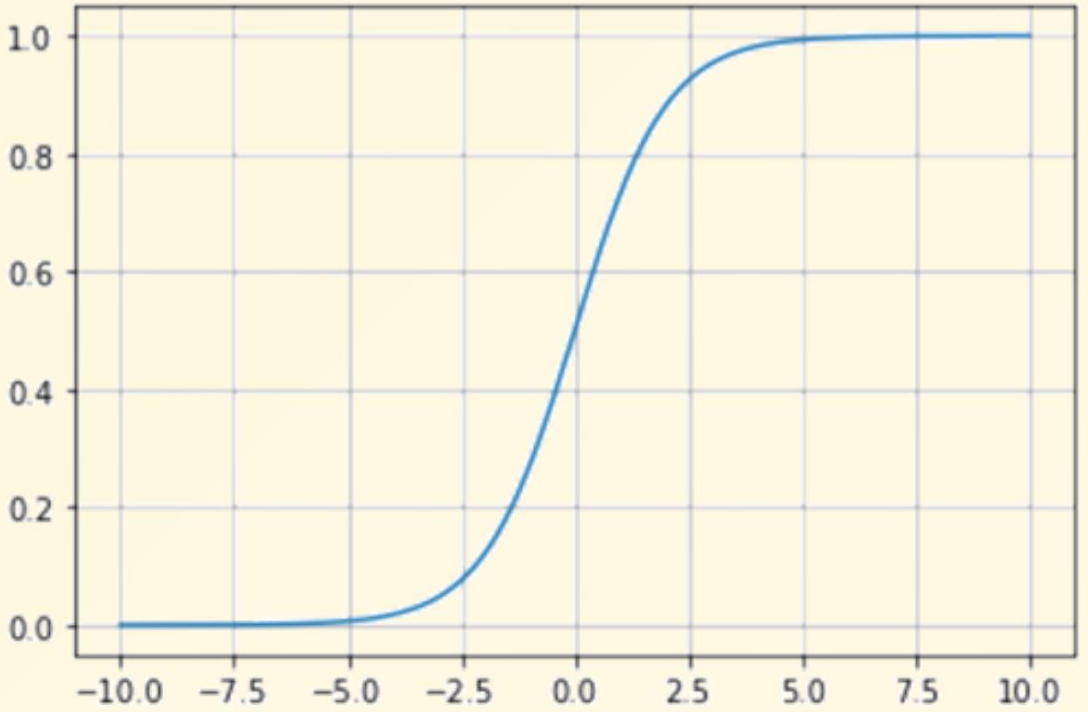
$$z = -\ln(\frac{1}{y} - 1)$$

$$e^{-z} = \frac{1}{y} - 1 \rightarrow e^{-z} = \frac{1-y}{y} \rightarrow y * (e^{-z} + 1) = 1$$

$$\rightarrow y = \frac{1}{1 + e^{-z}}$$

log-Odds

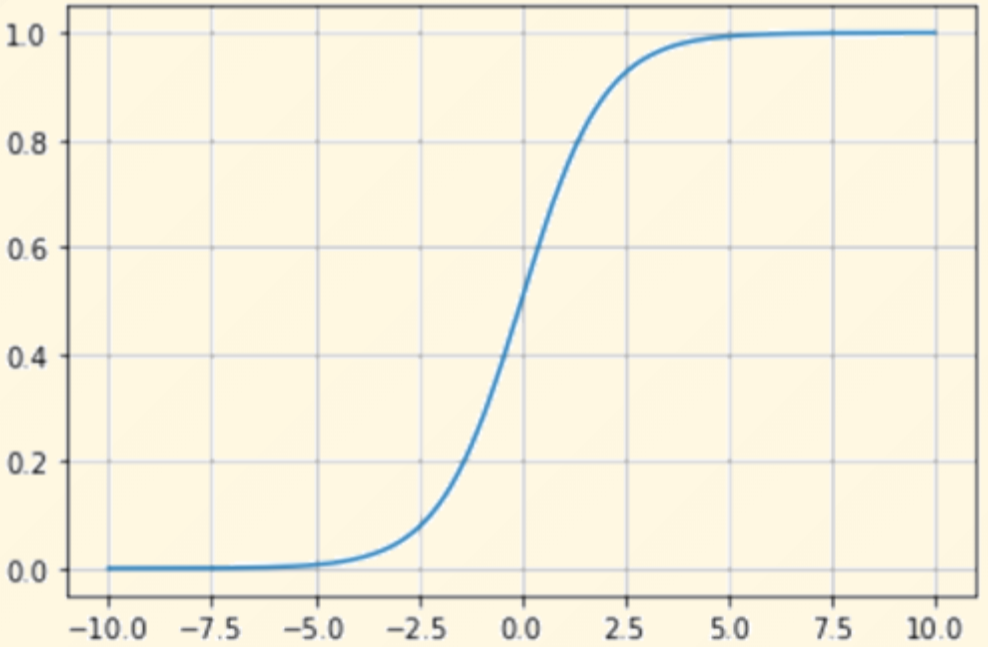
- **Inverse of logit function (Logistic Function)**
- S형태를 닮았다고 하여, **Sigmoid function**으로 불림
- $y = \dfrac{1}{1 + e^{-z}}$



이진 분류의 가설 함수

- 선형 회귀 모델에 **Sigmoid function**을 씌움

- $y = \dfrac{1}{1 + e^{-z}}$
- $z = \mathbf{w}^T \mathbf{x} + b$
- $y = \dfrac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$



이진 분류의 비용 함수

$$y_{pred} = H(\mathbf{x}) = \frac{1}{1+e^{-(\mathbf{w}^T\mathbf{x}+b)}}, \quad y_{true} = 0 \text{ or } 1$$

$$\text{cost}(y_{pred}, y_{true}) = \begin{cases} -\log(y_{pred}), & \text{if } y_{true} = 1 \\ -\log(1-y_{pred}), & \text{if } y_{true} = 0 \end{cases}$$

이진 분류의 비용 함수

$$\text{cost}(H(\mathbf{x}), y_{true}) = -\frac{1}{N}\sum_{i=1}^{N}[y_{true}^{(i)} \log H(\mathbf{x}) + (1 - y_{true}^{(i)}) \log(1 - H(\mathbf{x}))]$$

$$\text{where } H(\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}}$$

비용 함수의 경사하강법

$$\frac{\partial}{\partial w_j}\,\mathrm{cost}(\mathbf{w}) = \frac{\partial}{\partial w_j}\left[-\frac{1}{N}\sum_{i=1}^{N}\left[y^{(i)}\log(\sigma(z^{(i)})) + (1-y^{(i)})\log(1-\sigma(z^{(i)}))\right]\right]$$

$$= -\frac{1}{N}\sum_{i=1}^{N}\left[\frac{y^{(i)}}{\sigma(z^{(i)})} - \frac{1-y^{(i)}}{1-\sigma(z^{(i)})}\right]\sigma'(z^{(i)})x_j^{(i)}$$

$$= -\frac{1}{N}\sum_{i=1}^{N}(y^{(i)} - \sigma(z^{(i)}))x_j^{(i)}$$

비용 함수의 경사하강법

$$\text{cost}(H(\mathbf{x}), y_{true}) = -\frac{1}{N}\sum_{i=1}^{N}[y_{true}^{(i)} \log H(\mathbf{x}) + (1 - y_{true}^{(i)}) \log(1 - H(\mathbf{x}))]$$

$$\frac{\partial}{\partial w_j} \text{cost}(H(\mathbf{x}), y_{true}) = \frac{1}{N}\sum_{i=1}^{N}(H(\mathbf{x}) - y^{(i)}) \cdot x_j^{(i)}$$

$$w_{new} := w_{old} - \alpha \frac{\partial}{\partial w} \text{cost}(H(\mathbf{x}), y_{true})$$

- **이진 분류의 비용 함수도 경사하강법으로 업데이트가 가능하다!**

이진 분류의 최적화 (경사하강법)

$$w_{j,new} := w_{j,old} - \alpha \frac{\partial}{\partial w_j} \text{cost}(\mathbf{w}, b)$$

$$w_{j,new} := w_{j,old} - \alpha \frac{1}{N} \sum_{i=1}^{N} (H(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$b_{new} := b_{old} - \alpha \frac{1}{N} \sum_{i=1}^{N} (H(\mathbf{x}^{(i)}) - y^{(i)})$$

$$\text{where } H(\mathbf{x}) = \frac{1}{1+e^{-(\mathbf{w}^T \mathbf{x}+b)}}$$

아래와 같이 실습해보자!

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

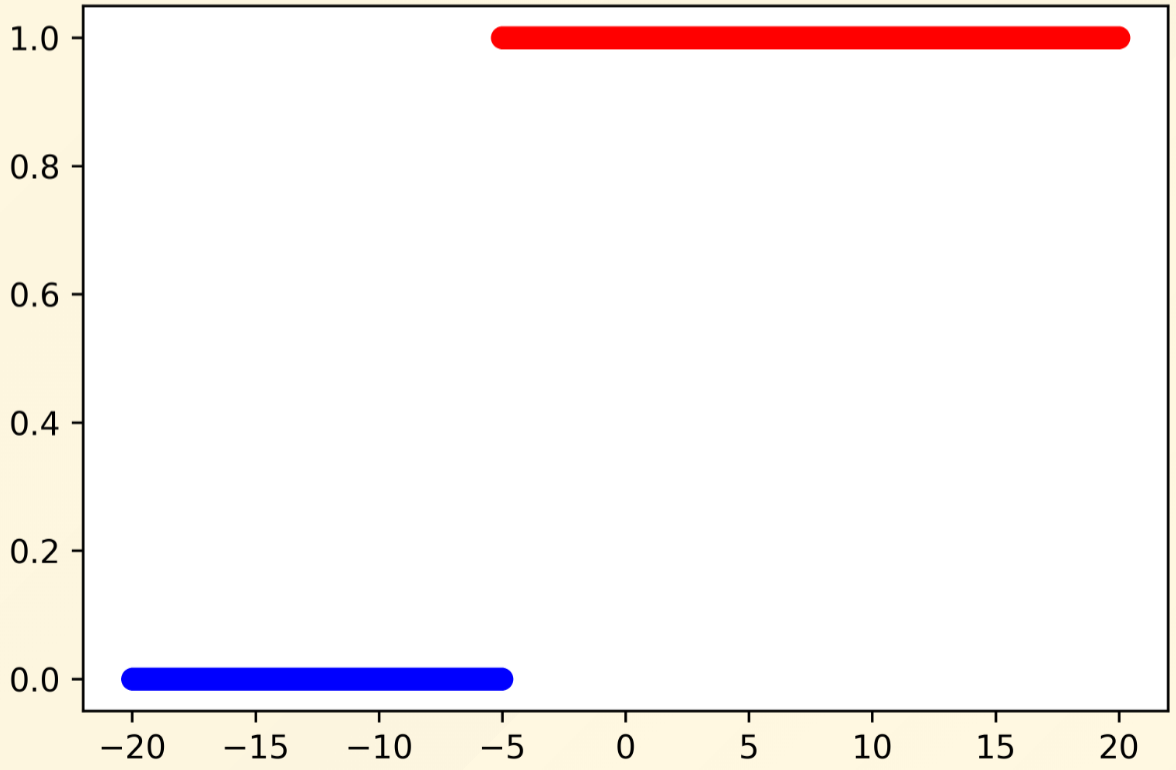
def hypothesis(w, x, b):
    return sigmoid(w * x + b)

def cost_function(h, y):
    return -np.mean(y * np.log(h + 1e-8) + (1.0 - y) * np.log(1.0 - h + 1e-8))
```

```
N = 5000
x = 40.0 * np.random.rand(N) - 20.0 # range : -20 ~ 20

# y = wx + b = 1.0*x + 5.0
z = 1.0 * x + 5.0
z = sigmoid(z)
y = np.where(z > 0.5, 1.0, 0.0)
idx = np.where(z > 0.5, True, False)
```

```
plt.plot(x[idx == True], y[idx == True], linestyle='None', marker='o', color='red')
plt.plot(x[idx == False], y[idx == False], linestyle='None', marker='o', color='blue')
plt.show()
```



```
x_train = x[:4000]
y_train = y[:4000]
x_test = x[4000:]
y_test = y[4000:]
print(x_train.shape, y_train.shape, x_test.shape, y_test.shape)
```

→ `(4000,) (4000,) (1000,) (1000,)`

```
w = np.random.rand()
b = np.random.rand()
print(w, b)
```

→ `0.9755818034895956 0.07271016164937016`

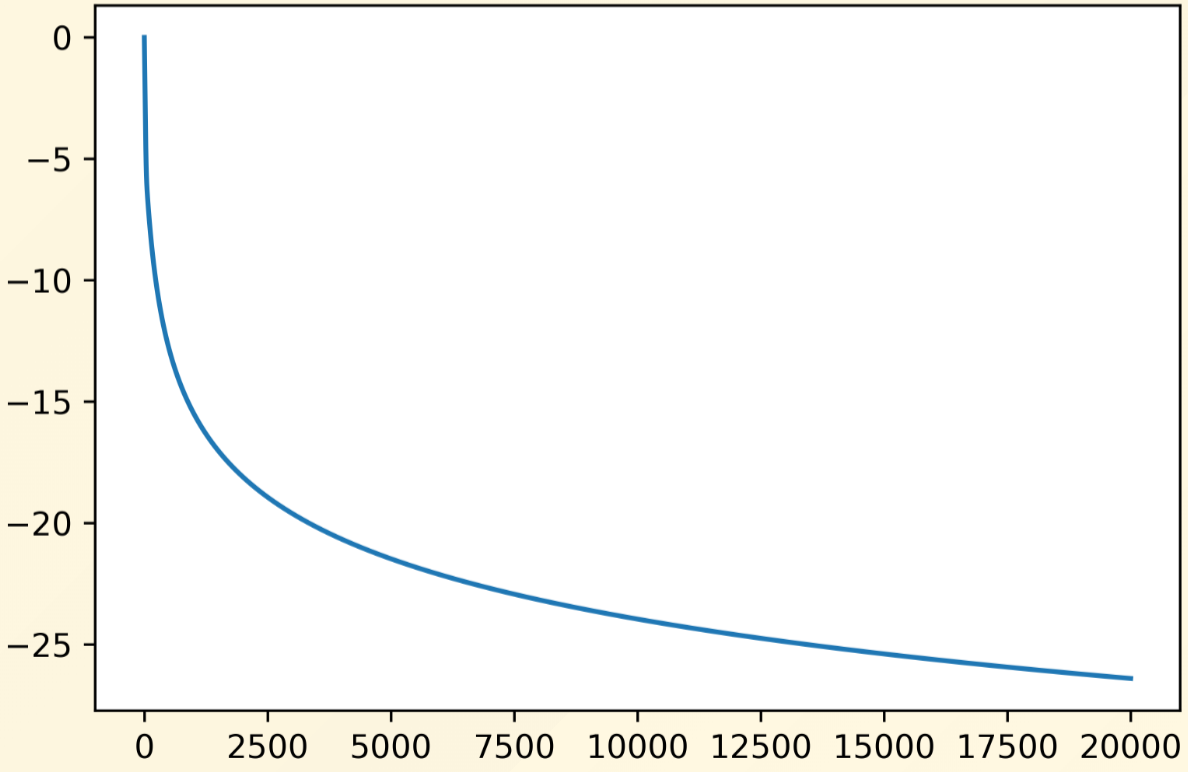
```
epoch = 20000
alpha = 6e-2

total_loss = []
for i in range(epoch):
    h = hypothesis(w, x, b)
    loss = cost_function(h, y)

    w = w - alpha * np.mean((h - y) * x)
    b = b - alpha * np.mean((h - y))

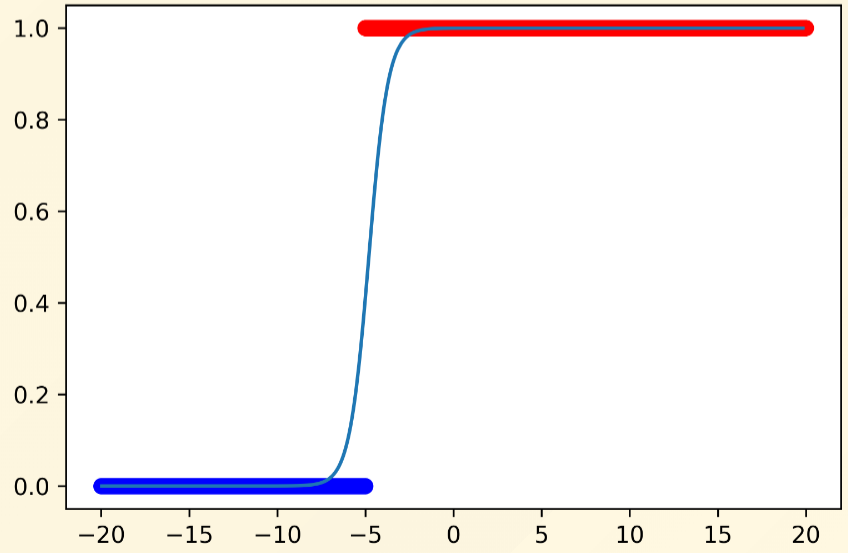
    total_loss.append(loss)
total_loss = np.array(total_loss)
```

```
plt.plot(10.0 * np.log(total_loss / (np.max(total_loss + 1e-5))))
plt.show()
```



```
x_test_sorted = np.sort(x_test)
y_pred = hypothesis(w, x_test_sorted, b)

plt.plot(x[idx == True], y[idx == True], linestyle='None', marker='o', color='red')
plt.plot(x[idx == False], y[idx == False], linestyle='None', marker='o', color='blue')
plt.plot(x_test_sorted, y_pred)
plt.show()
```



```
from sklearn.metrics import accuracy_score

y_pred = hypothesis(w, x_test, b)
y_pred = np.round(y_pred)

acc = accuracy_score(y_test, y_pred)
print(acc)
```

→ `0.995`